

Uncertainty Quantification on the Friedman Function

Tara Welch

January 2026

1 Introduction

Complex physical models are often too computationally expensive for thorough analysis, sometimes taking days or weeks to run. To address this, Bayes Linear emulators can be used as statistical surrogate functions, as they are much faster to evaluate.

The following project will explore the Friedman function [1]:

$$f(\mathbf{x}) = 10 \sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5, \quad (1)$$

with $\mathbf{x} \in [0, 1]^5$, so no further scaling is required. For this project this function is implemented in R, sourced from [3]. Although the Friedman function does not represent a physical model, it acts as a benchmark for testing emulator performance, such as in the R package *mlbench* [2]. Researchers can use this to ensure their emulators can handle non-linearities and interactions, before deploying them on computationally expensive simulations.

1.1 Input Parameters

- The function is symmetric in x_1 and x_2 . The maximum value of the first term is 10, occurring when the product $x_1 x_2$ is equal to 0.5. This term introduces non-linearity to the function due to using sine.
- x_3 has the largest coefficient, which could indicate a greater effect on the output, however due to the domain and centering term $(x_3 - 0.5)$, the effective range is restricted so that this term has a maximum contribution of 5. This term introduces quadratic non-linearity, and is symmetric about the global minimum, $x_3 = 0.5$.
- The x_4 and x_5 terms contribute linearly with maxima of 10 and 5 respectively.
- x_1 and x_2 present coupling, since the sensitivity of the output to one variable is dependent on the value of the other. For this reason, they will be explored in 2D emulation and history matching.
- As seen in fig. 1e, at low values of x_1 , the effect on function output of changing x_2 is minimal, however as x_1 increases towards 1 this effect increases dramatically (and vice versa for x_2). This can be seen clearly by comparing the contour along the axes (flat, single-valued), to the contour along $x_1 = 1$ or $x_2 = 1$ (varied).
- In fig. 1f we see the impact of the coefficient of 20, as approaching $x_3 = 0$ or $x_3 = 1$ the contour reaches the maximum value. The x_3 term dominates the x_1 term here, indicated by the plot largely following parabolic shape as in fig. 1b.

2 1D Emulation

- We now conduct 1D emulation of the variable x_3 , fixing all other inputs at 0.5. This variable was chosen to explore the non-linearity, and also because it has significant effect on the output.
- To replicate the constraints of a simulator, the true function is modelled as being computationally expensive. Therefore the experimental design is restricted to a minimal number of runs ($N=6$).
- We perform 6 runs, equally spaced between 0 and 1. This ensures uniform coverage of the domain.
- By performing runs on the boundary points $x_3 = 0, x_3 = 1$, we eliminate extrapolation so that emulation is only performed between runs. This keeps variance bounded.
- We see the variance is 0 at the design points, and grows to a maximum halfway between these points, as expected.

The emulator performance is governed by two hyperparameters, the correlation length (θ) and the prior variance (σ^2).

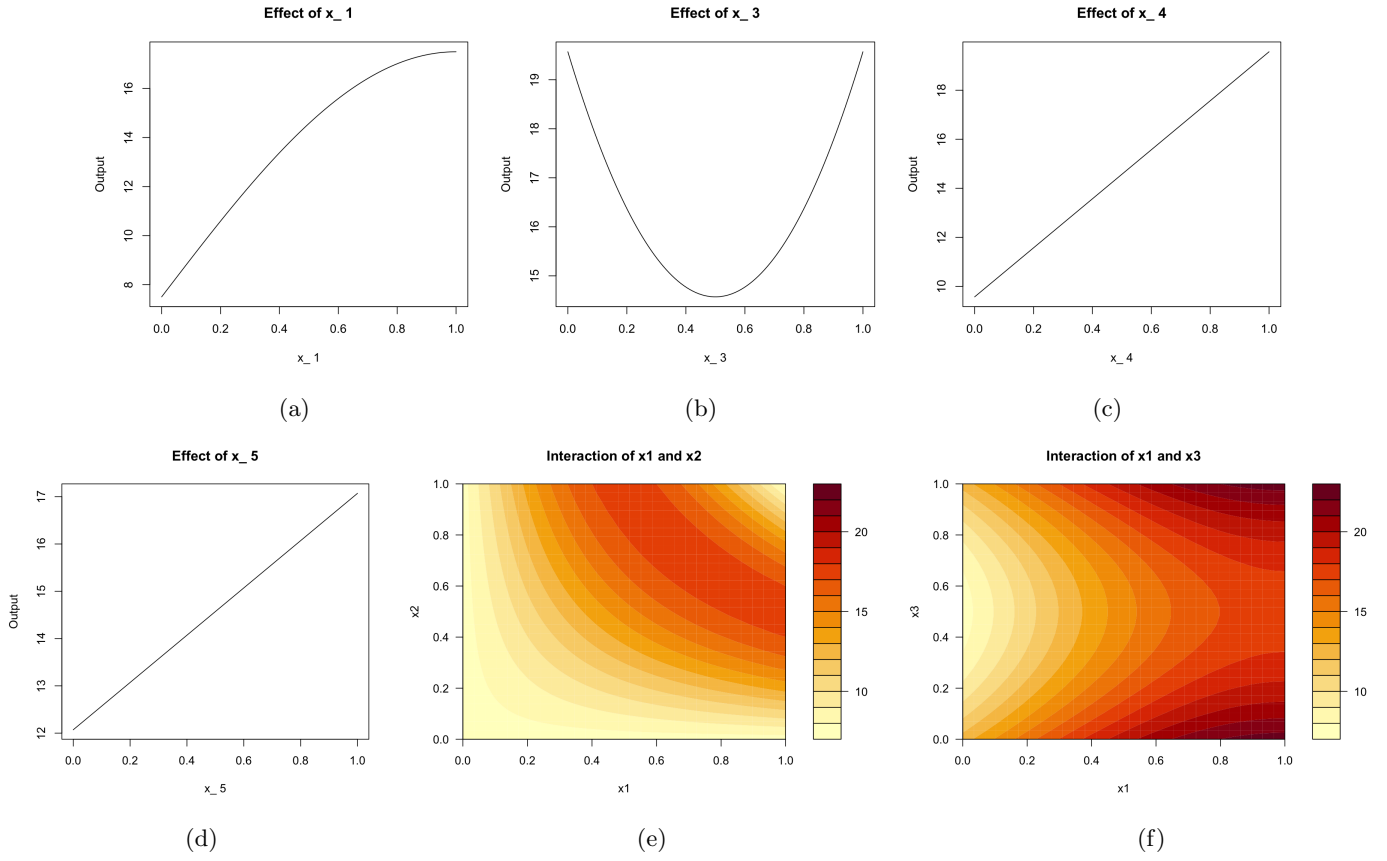


Figure 1: Subfigures (a)-(d) show each variable plotted against the output $f(\mathbf{x})$, with all other variables fixed at a value of 0.5. Note that the effect of x_2 is identical to (a) so this has been omitted to avoid repetition. Subfigures (e) and (f) show the contour plots of x_1 with x_2 and x_3 respectively, again with all other variables fixed at 0.5.

2.1 Tuning Prior Variance (σ^2)

- Changing prior variance (σ^2) changes the emulator standard deviation, without affecting the emulator expectation. It essentially scales how confident we are that the true function is close to the emulator expectation.
- When σ is small, this produces a narrower prediction interval, essentially resulting in overly confident predictions. As seen in fig. 2a, for σ too small the true function may fall outside the emulator prediction interval (e.g. $x_3 < 0.2$).
- When σ is large, the bounds are conservative, however this is uninformative as it underestimates our knowledge of the function's behaviour (fig. 2d).
- Based on fig. 2, and with the aim of balancing the extremes explained above, I decided that a good choice was $\sigma = 2$.

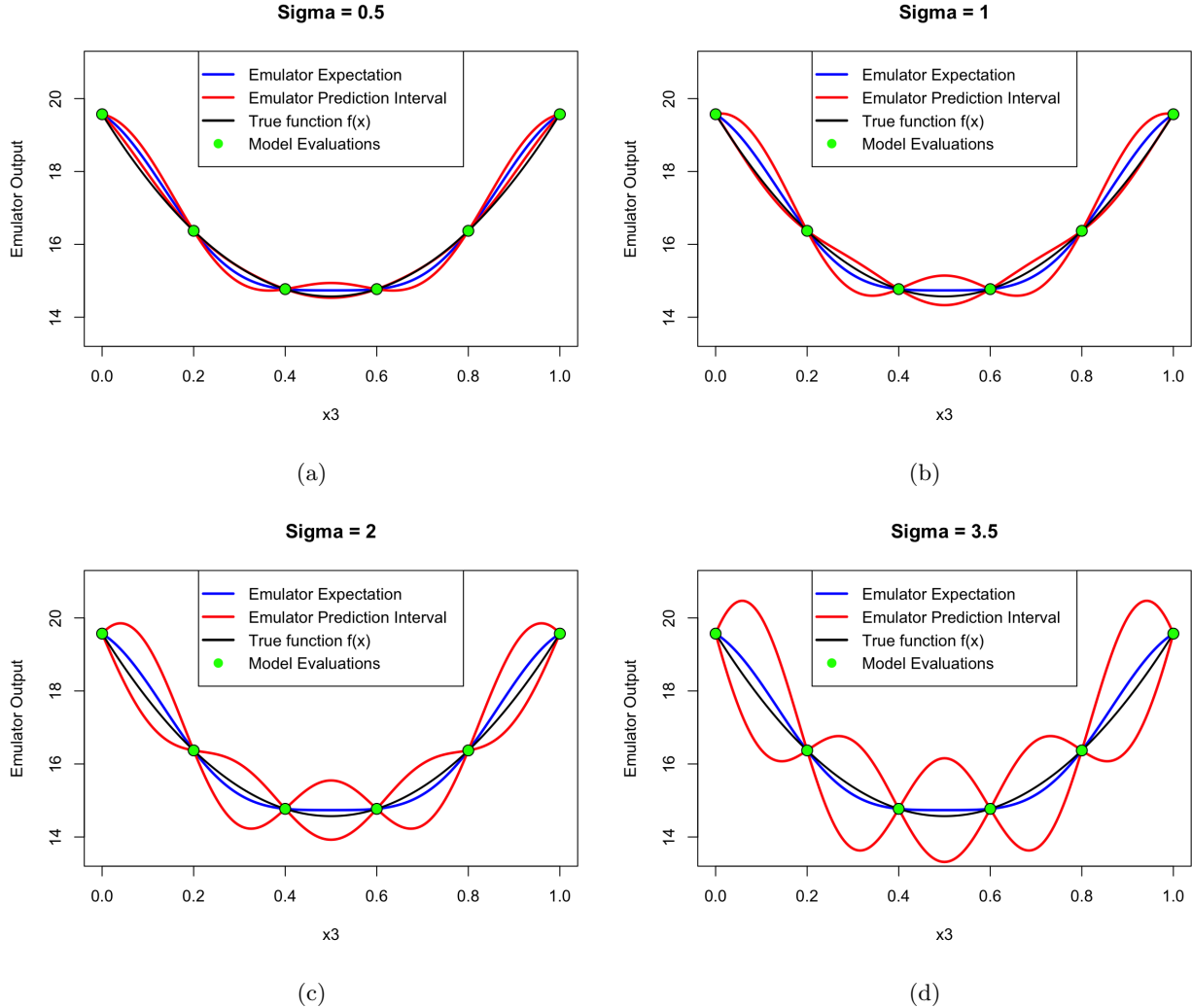


Figure 2: 1D emulator eq. (1), with $x_1 = x_2 = x_4 = x_5 = 0.5$, and varying x_3 over its domain $[0, 1]$. Demonstration of the effect of different values of sigma, with $\sigma = 0.5$ (a), $\sigma = 1$ (b), $\sigma = 2$ (c) and $\sigma = 3.5$ (d).

2.2 Tuning Correlation Length

- Changing correlation length changes the distance at which points influence one another. Essentially, the emulator expects points that are close to each other to have similar outputs. The correlation length quantifies how close the points need to be, in other words what we consider a 'neighbourhood'.
- When θ is too low, the emulator produces spikes (fig. 3a) as the information from a design point is not effectively shared with neighbouring points. This means the prediction reverts to the prior mean between data points, rather than interpolating.

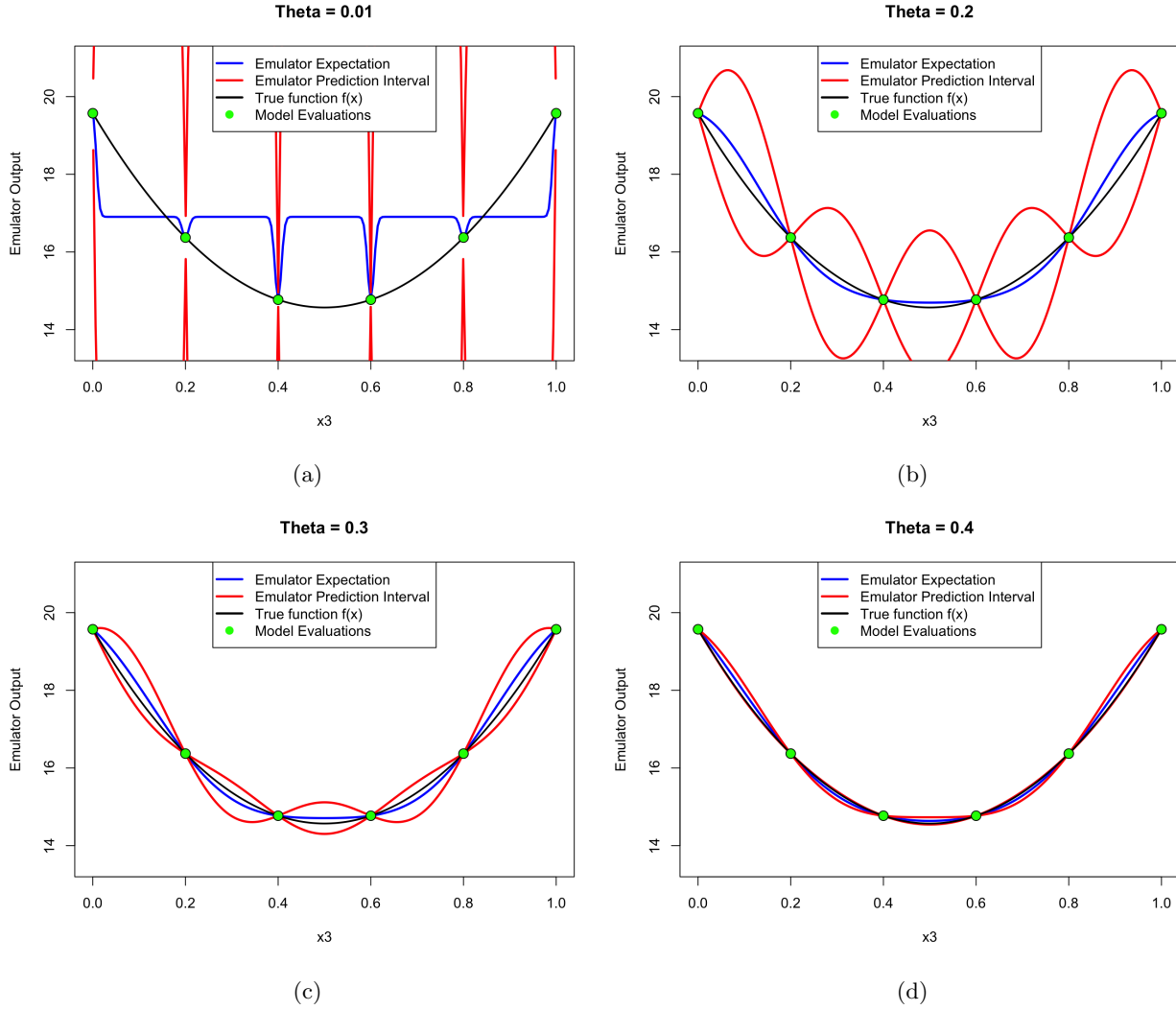


Figure 3: 1D emulator eq. (1), with $x_1 = x_2 = x_4 = x_5 = 0.5$, and varying x_3 over its domain $[0, 1]$. Demonstration of the effect of different values of theta, with $\theta = 0.01$ (a), $\theta = 0.2$ (b), $\theta = 0.3$ (c) and $\theta = 0.4$ (d).

- When θ is too high, the emulator assumes the function is entirely smooth, with correlation extending across the entire domain. This can over-simplify more complex functions, though in this case was relatively accurate (fig. 3d).
- Based on fig. 3, I decided that $\theta = 0.3$ was a good choice, since it uses the data to interpolate whilst remaining flexible and not over-smoothing.
- Since θ and σ interact, I set sigma at 2 as decided above, in order to produce the plots fig. 3.

We see that in fig. 2 and fig. 3 for the chosen values of parameters $\sigma = 0.2$ and $\theta = 0.3$, the true function lies within the emulator prediction interval, indicating it is within 3 standard deviations of the emulator expectation. In these cases, the emulator behaves as expected with the prediction interval collapsing at observed data points, and ballooning out as it moves away from these, reaching its maximum at the midpoint between two design points where our confidence is lowest.

3 2D Emulation

- We now conduct 2D emulation. Due to the complexity added by the interaction between x_1 and x_2 , these variables are selected as our 2 dimensions of interest, with the other three fixed at their midrange value, 0.5.
- A Latin Hypercube Design is used to allocate 20 well-spaced input points.

- We iterate over different values of θ and analyse the shape of the emulator adjusted expectation, as well as adjusted variance, plotted in fig. 4.
- The plots on the left side of fig. 4 demonstrate the change in adjusted expectation as θ increases. For low θ , each point is isolated and the expectation reverts to the mean in the gaps. As θ increases, we see the ridge structure of the interaction and the expectation takes the shape of the true function (see fig. 1e).
- The right-side plots of fig. 4 show the effect of θ on adjusted variance of the emulator. Again, for low values of θ , the emulator shows the points as disconnected, with high uncertainty between points. For higher values of θ , the variance is low throughout, with the maximum variance dropping from around 12 to less than 3 almost everywhere. This may indicate overconfidence, as points such as $(1, 1)$ have low uncertainty despite no design point nearby, putting the emulator at risk of over-smoothing.
- To strike a balance between capturing the shape of the function, but also maintaining variance in unexplored regions, a value of $\theta = 0.3$ would be a good choice here.
- Note that a higher correlation length (e.g. fig. 4g) produces an expectation closer to the shape of the true function in this case. This is because the underlying ‘ $10 \sin(\pi x_1 x_2)$ ’ term is smooth. However in practical applications of an emulator, the underlying function may not be known (black box scenario) and so assuming this smoothness can be a risk.
- Aside from comparing the shape, we can also verify the quantitative accuracy of the emulator. The theoretical range is $f(x_1, x_2, 0.5, 0.5, 0.5) = 10 \sin(\pi x_1 x_2) + 7.5 \in [7.5, 17.5]$. This is accurately reflected in the emulator’s adjusted expectations which have maxima and minima closely matching these bounds.

4 2D History Matching

We perform 2D history matching on the pairs (x_1, x_2) and then to (x_1, x_3) , each with two waves, again with other inputs set to 0.5.

4.1 Wave 1 (x_1 and x_2)

- We set the observation to be $z = 14.5$ to test the emulator performance on 2 regions of non-implausibility due to symmetry of the *sin* term.
- The first wave consisted of 14 points in a Latin Hypercube design. While it successfully rules out large regions, the non-implausible region remains large and connected (fig. 5a).

4.2 Wave 2 (x_1 and x_2)

- The second wave consists of 10 runs within the non-implausible region defined by wave 1.
- fig. 5b shows the implausibility after only 2 second wave points are evaluated, emphasising the impact of these second wave points on emulator belief.
- fig. 5c is the result of the second wave, showing the emulator successfully forming regions that encapsulate the true regions (fig. 5d), and are of the correct shape but slightly wider to account for emulator variance.
- 10 runs were chosen due to an iterative approach, adding runs sequentially to ensure sufficient coverage of the region found in wave 1 without unnecessary computational cost.

4.3 Comparison with x_1 and x_3

- For comparison, a history match of x_1 and x_3 was performed. Due to the simpler topology of the non-implausible region, since x_1 and x_3 terms are functionally independent (additive), the history match was an even tighter match to the true boundary implausibility for the same number of runs (fig. 6).

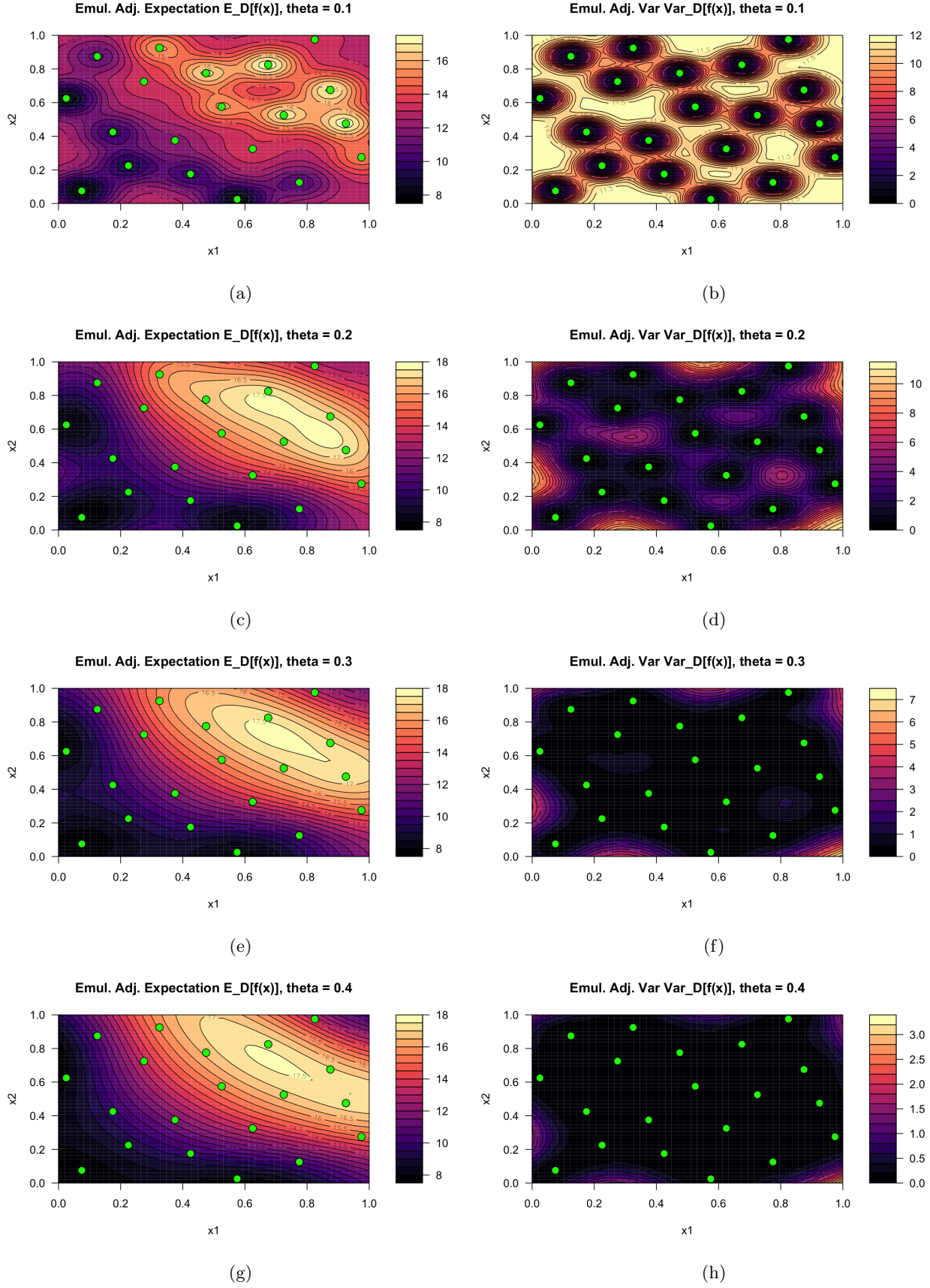


Figure 4: 2D emulation of eq. (1), with $x_3 = x_4 = x_5 = 0.5$, and varying x_1 and x_2 over their domain, $[0, 1]^2$. Emulator adjusted expectation (left) and emulator adjusted variance (right) shown for $\theta=0.1, 0.2, 0.3, 0.4$.

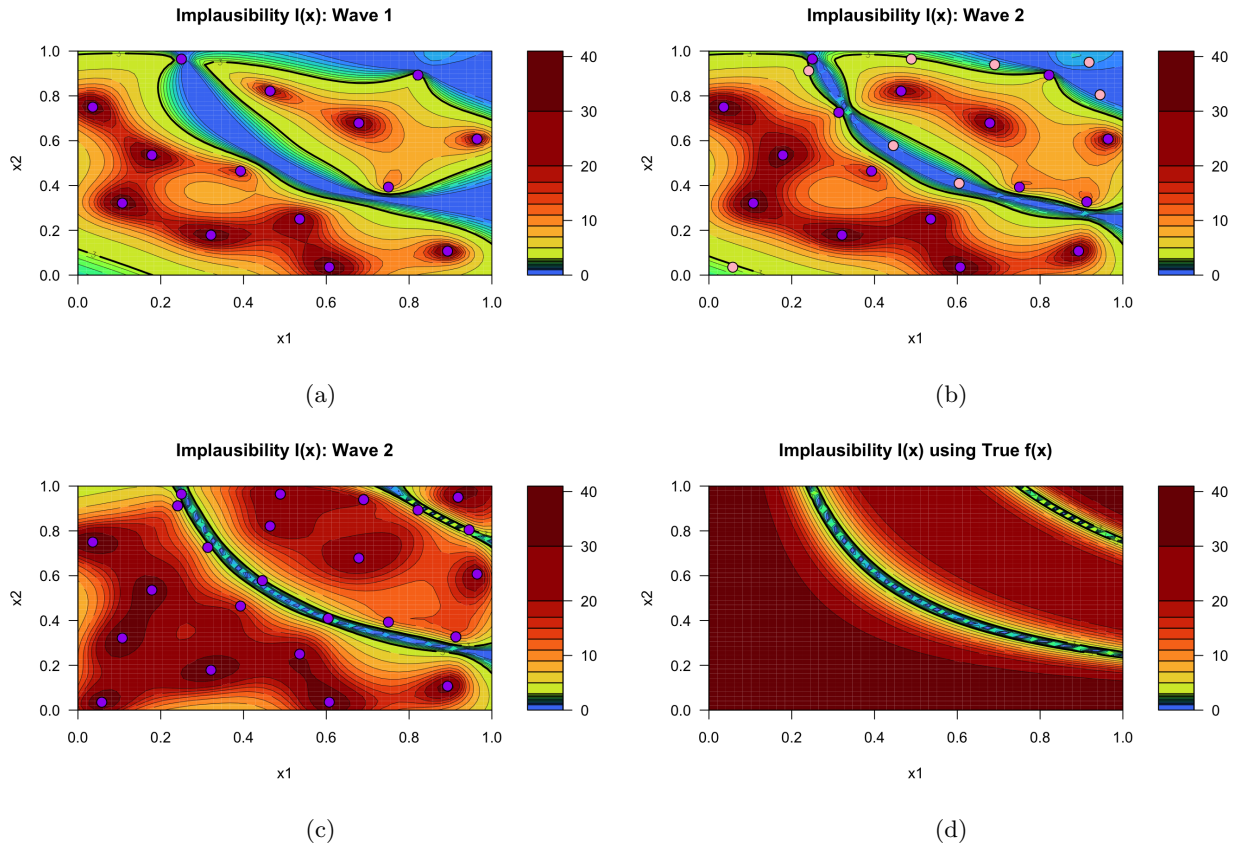


Figure 5: Implausibility plot for wave 1 (a) and wave 2 (b),(c) of history matching x_1 and x_2 , with $z = 14.5$.

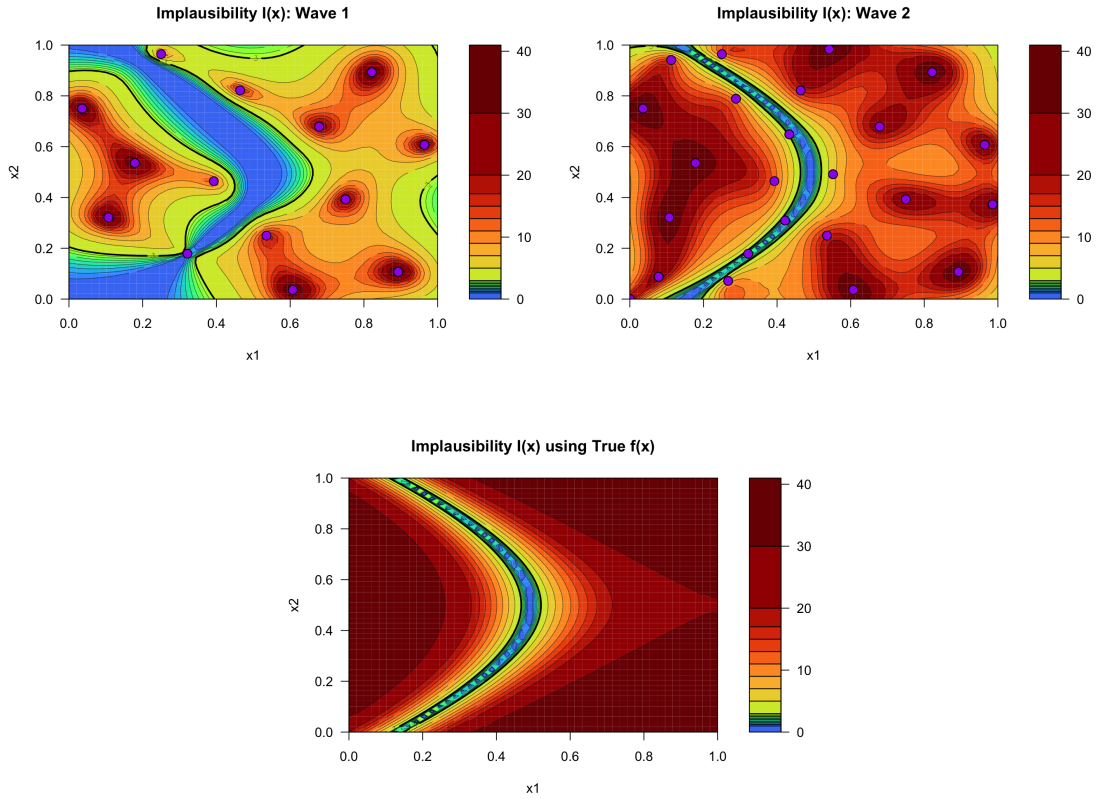


Figure 6: Implausibility plots for two wave history matching for x_1 and x_3 , with $z = 14.5$.

5 Optimisation

We can approach optimisation with a history matching style strategy to find a set of inputs that maximise our function $f(\mathbf{x})$. For this, to obtain interesting plots, we again focus on the variables (x_1, x_2) , since the quadratic and linear terms are simply maximised at their boundaries. To investigate the global max, we set $x_3 = x_4 = x_5 = 1$.

- Wave 1 is shown in fig. 7a, which forms a single connected region, unlike the disjoint arcs in the history matching case (fig. 5).
- Wave 1 alone is highly effective, capturing the approximate location of the maximum function value.
- Given the success of wave 1, wave 2 required fewer additional points. Again points were added sequentially to tighten the boundaries, producing a final plot fig. 7c closely matching the true values fig. 7d.
- Inspection of the emulator adjusted expectation confirmed the accuracy of the magnitude of the peak. The emulator predicted a maximum of $E_D[f(x)] \approx 29.5$ after wave 1 (fig. 8a) and obtained the theoretical maximum of 30 in some regions after wave 2 (fig. 8b).

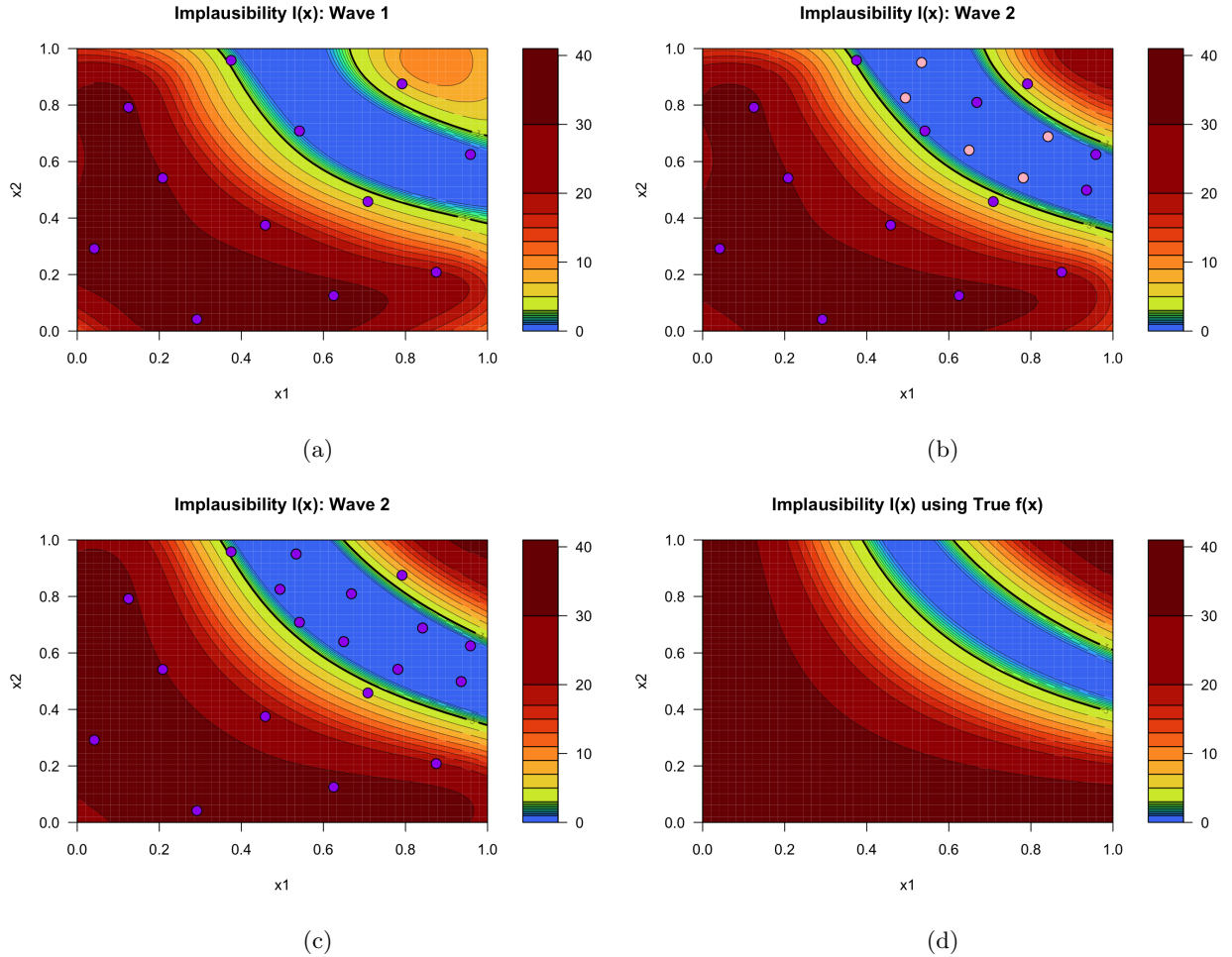


Figure 7: Implausibility plots of optimisation of eq. (1) wrt x_1 and x_2 , with wave 1 (a), part-way through wave 2 (b), end of wave 2 (c) and comparison with true function (d). Fixed $x_3 = x_4 = x_5 = 1.0$.

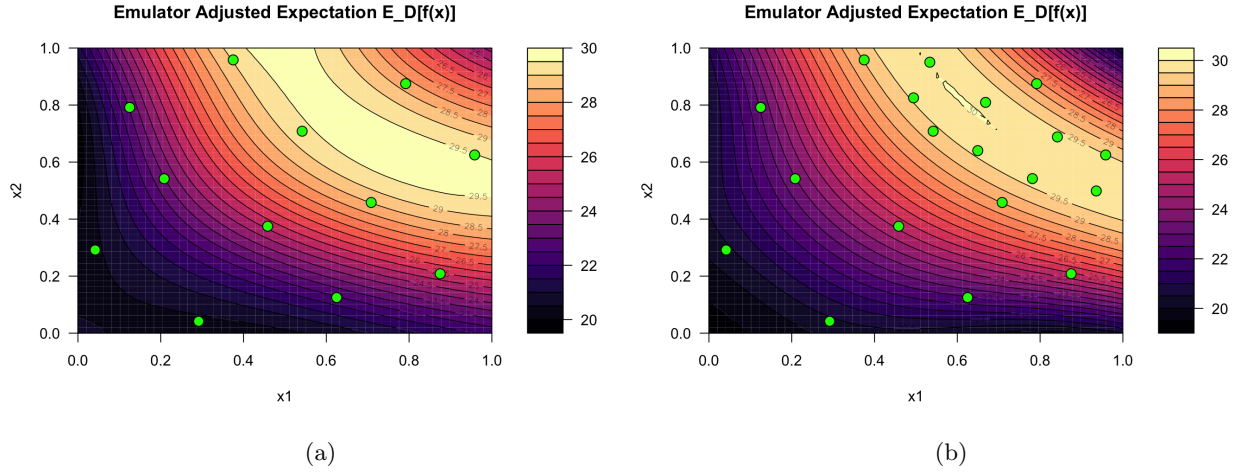


Figure 8: Emulator adjusted expectation after optimisation of eq. (1) wrt x_1 and x_2 after wave 1 (a) and wave 2 (b).

6 Conclusion

This project explored the use of simple Bayes Linear emulators on the non-linear Friedman function.

- Tuning of sigma and theta was performed, highlighting its influence on ensuring the emulator expectation follows the shape of the true function, and to obtain sufficiently narrow prediction intervals without making overly confident predictions.
- History matching of additive terms (x_1 vs x_3) may only require a single wave for accurate results, whereas for interaction terms (x_1 vs x_2), 2 waves were required to emulate the region successfully.
- In all cases of history matching, the emulator's non-improbable region successfully encapsulated the true model contours. This is important as it ensures there are no false negatives.
- It is important to acknowledge that the Friedman function is smooth, and so emulator behaviour may represent a best case scenario. A more rigorous stress-test could include non-smooth, discontinuous functions, possibly with multiple interaction terms.

Overall this report demonstrated techniques for exploring complex, non-linear models, using an emulator as a robust, low-cost surrogate. The BL emulators were able to characterise surfaces of parameter interaction as well as identify global optima, using iterative design to ensure low computational cost.

References

- [1] J.H. Friedman. Multivariate adaptive regression splines. *The Annals of Statistics*, 1991.
- [2] Evgenia Dimitriadou Friedrich Leisch and Kurt Hornik. *mlbench: Machine Learning Benchmark Problems*, 2024. R package version 2.1-5.
- [3] S. Surjanovic and D. Bingham. Virtual library of simulation experiments: Test functions and datasets, 2013. Accessed: 2026-01-09.